CORE CSE SERIES — MCQ EDITION

DBMS MCQ Handbook

Basic to Advanced Multiple-Choice Questions for Company OTs, PYQ-style Practice & Core Placement Preparation — with Answers & Explanations

ER & Keys | Normalization | SQL & Joins | Transactions | Indexing | NoSQL & Distributed

Compiled for Product & Service-based Company Online Tests, GATE-style Core Practice, and Interview Rounds

Table of Contents — DBMS

DBMS

Topic-wise MCQs with Explanations

1. ER Model, Keys & Relational Model

Entities, relationships, candidate/primary/foreign keys

Q1 **What is a candidate key?** BASIC

A. A minimal set of attributes that can uniquely identify a tuple in a relation

B. Any attribute in a table

C. A key used only for foreign relations

D. A key that must be numeric

Correct Answer: A. A minimal set of attributes that can uniquely identify a tuple in a relation

Explanation: A candidate key is a minimal super key — no attribute can be removed without losing the uniqueness property. A table may have multiple candidate keys; one is chosen as the primary key.

Q2 **A foreign key is used to:** BASIC

A. Establish and enforce a link between data in two tables

B. Uniquely identify a row in the same table

C. Store encrypted data

D. Speed up all queries automatically

Correct Answer: A. Establish and enforce a link between data in two tables

Explanation: A foreign key in one table references the primary key of another table, enforcing referential integrity between related tables.

Q3 **Which of the following best describes a 'weak entity' in ER modeling?** INTERMEDIATE

A. An entity that cannot be uniquely identified by its own attributes alone and depends on a related 'owner' entity

B. An entity with no attributes

C. An entity with only one relationship

D. An entity that only exists in NoSQL databases

Correct Answer: A. An entity that cannot be uniquely identified by its own attributes alone and depends on a related 'owner' entity

Explanation: A weak entity lacks a sufficient primary key of its own and relies on a foreign key combined with a partial key (discriminator) from its owning strong entity for unique identification.

Q4 **In ER diagrams, a double-lined rectangle typically represents:** INTERMEDIATE

A. A weak entity set

B. A strong entity set

C. An attribute

D. A relationship

Correct Answer: A. A weak entity set

Explanation: By ER diagram convention, a double-lined rectangle denotes a weak entity, while a double-lined diamond denotes its identifying relationship.

Q5 **What does 'referential integrity' ensure in a relational database?** INTERMEDIATE

A. A foreign key value must either match an existing primary key value or be NULL

B. All tables must have the same number of rows

C. All primary keys must be integers

D. Data can never be deleted

Correct Answer: A. A foreign key value must either match an existing primary key value or be NULL

Explanation: Referential integrity prevents 'orphan' foreign key values that don't correspond to any valid row in the referenced table.

Q6 **A relation is said to be in violation of entity integrity if:** ADVANCED

A. A primary key attribute contains a NULL value

B. A foreign key references a non-existent row

C. Two rows have identical non-key attributes

D. A table has more than one candidate key

Correct Answer: A. A primary key attribute contains a NULL value

Explanation: Entity integrity requires that no attribute of the primary key can be NULL, since NULL cannot guarantee uniqueness.

2. Normalization 1NF, 2NF, 3NF, BCNF, functional dependencies

Q7 **A relation is in 1NF if:** BASIC

A. All attribute values are atomic (indivisible), with no repeating groups
B. It has no foreign keys
C. It has only one candidate key
D. It has been indexed

Correct Answer: A. All attribute values are atomic (indivisible), with no repeating groups

Explanation: 1NF requires that each column hold only atomic, single values, and each row be uniquely identifiable, with no repeating groups/arrays within a cell.

Q8 **A relation violates 2NF when:** INTERMEDIATE

A. A non-key attribute depends on only part of a composite primary key (partial dependency)
B. It is not in 1NF
C. A non-key attribute depends on another non-key attribute
D. It has more than 3 attributes

Correct Answer: A. A non-key attribute depends on only part of a composite primary key (partial dependency)

Explanation: 2NF requires full functional dependency — every non-key attribute must depend on the ENTIRE composite primary key, not just part of it. This only matters when the PK is composite.

Q9 **A relation violates 3NF when there is:** INTERMEDIATE

A. A transitive dependency (a non-key attribute depends on another non-key attribute)
B. A partial dependency on the primary key
C. Atomic values only
D. No foreign keys defined

Correct Answer: A. A transitive dependency (a non-key attribute depends on another non-key attribute)

Explanation: 3NF eliminates transitive dependencies — non-key attributes must depend directly on the primary key, not indirectly through another non-key attribute.

Q10 **BCNF (Boyce-Codd Normal Form) is a stricter version of 3NF because it requires:** ADVANCED

A. For every functional dependency $X \rightarrow Y$, X must be a super key
B. Every attribute to be a foreign key
C. No composite keys allowed
D. Only single-column tables

Correct Answer: A. For every functional dependency $X \rightarrow Y$, X must be a super key

Explanation: BCNF requires that the left-hand side (determinant) of every non-trivial functional dependency must be a super key, closing edge cases 3NF doesn't fully cover.

Q11 **What is a functional dependency X → Y?** BASIC

A. The value of X uniquely determines the value of Y

B. X and Y are always independent

C. X must equal Y

D. X is a foreign key referencing Y

Correct Answer: A. The value of X uniquely determines the value of Y

Explanation: X → Y means that for any two tuples with the same value of X, the value of Y must also be the same — X functionally determines Y.

Q12 **What is the main goal of database normalization?** BASIC

A. To reduce data redundancy and avoid update/insert/delete anomalies

B. To increase the number of tables regardless of redundancy

C. To make all queries use joins

D. To eliminate the need for primary keys

Correct Answer: A. To reduce data redundancy and avoid update/insert/delete anomalies

Explanation: Normalization organizes data to minimize redundancy and prevent anomalies, though it can be intentionally relaxed (denormalized) for read performance in some systems.

3. SQL Queries & Joins SELECT, JOIN types, aggregate functions, subqueries

Q13 **Which SQL clause is used to filter groups after a GROUP BY, based on aggregate conditions?** BASIC

A. HAVING

B. WHERE

C. FILTER

D. GROUP FILTER

Correct Answer: A. HAVING

Explanation: WHERE filters rows before grouping; HAVING filters groups after aggregation (e.g., HAVING COUNT(*) > 5).

Q14 **Which JOIN returns only rows with matches in both tables?** BASIC

A. INNER JOIN
B. LEFT JOIN
C. RIGHT JOIN
D. FULL OUTER JOIN

Correct Answer: A. INNER JOIN

Explanation: INNER JOIN returns only the rows where the join condition is satisfied in both tables; unmatched rows from either side are excluded.

Q15 **What does a LEFT JOIN return that an INNER JOIN would not?** BASIC

A. All rows from the left table, with NULLs for unmatched columns from the right table
B. All rows from the right table only
C. Only unmatched rows
D. Duplicate rows only

Correct Answer: A. All rows from the left table, with NULLs for unmatched columns from the right table

Explanation: LEFT JOIN includes every row from the left table regardless of a match, filling right-table columns with NULL when there's no corresponding match.

Q16 **What is the difference between UNION and UNION ALL?** INTERMEDIATE

A. UNION removes duplicate rows; UNION ALL keeps all rows including duplicates
B. UNION is faster than UNION ALL always
C. UNION ALL requires matching column names; UNION does not
D. There is no difference

Correct Answer: A. UNION removes duplicate rows; UNION ALL keeps all rows including duplicates

Explanation: UNION performs an implicit DISTINCT to remove duplicate rows across the combined result sets, which is more expensive than UNION ALL, which simply concatenates results.

Q17 **A correlated subquery is one that:** ADVANCED

A. References a column from the outer query, so it's re-evaluated for each row of the outer query
B. Runs completely independently of the outer query
C. Can only appear in the WHERE clause
D. Must always return a single value

Correct Answer: A. References a column from the outer query, so it's re-evaluated for each row of the outer query

Explanation: A correlated subquery depends on the outer query's current row, so conceptually it executes once per outer row (though optimizers may rewrite it for efficiency).

Q18 **Which SQL keyword is used to remove duplicate rows from a result set?** BASIC

A. DISTINCT

B. UNIQUE

C. REMOVE DUPLICATES

D. FILTER

Correct Answer: A. DISTINCT

Explanation: The DISTINCT keyword in a SELECT statement eliminates duplicate rows from the returned result set.

Q19 **What does the SQL COALESCE() function do?** INTERMEDIATE

A. Returns the first non-NULL value from a list of expressions

B. Combines two tables

C. Converts a string to uppercase

D. Counts non-null rows

Correct Answer: A. Returns the first non-NULL value from a list of expressions

Explanation: COALESCE(a, b, c, ...) evaluates arguments in order and returns the first one that is not NULL — commonly used to supply default values.

Q20 **What is the output of `SELECT COUNT(*) FROM table` if the table has rows with some NULL values in a specific column?** INTERMEDIATE

A. COUNT(*) counts all rows regardless of NULLs in any column

B. It errors out

C. It only counts rows with no NULLs anywhere

D. It returns NULL

Correct Answer: A. COUNT(*) counts all rows regardless of NULLs in any column

Explanation: COUNT(*) counts all rows in the result set regardless of NULL values; COUNT(column_name) instead only counts non-NULL values in that specific column.

4. Transactions & Concurrency Control ACID, isolation levels, locking, deadlocks in DB

Q21 **Which ACID property ensures a transaction is either fully completed or fully rolled back?** BASIC

A. Atomicity
B. Consistency
C. Isolation
D. Durability

Correct Answer: A. Atomicity

Explanation: Atomicity guarantees a transaction is treated as a single indivisible unit — either all its operations succeed, or none do.

Q22 **Which ACID property ensures that once a transaction is committed, its changes survive even a system crash?** BASIC

A. Durability
B. Atomicity
C. Consistency
D. Isolation

Correct Answer: A. Durability

Explanation: Durability guarantees committed data is permanently saved (typically via write-ahead logging) and will not be lost even in the event of a power failure or crash.

Q23 **What is a 'dirty read'?** INTERMEDIATE

A. Reading data written by an uncommitted transaction, which might later be rolled back
B. Reading the same row twice with different results
C. Reading a row that has been deleted
D. Reading corrupted disk data

Correct Answer: A. Reading data written by an uncommitted transaction, which might later be rolled back

Explanation: A dirty read occurs when a transaction reads uncommitted changes from another transaction; if that transaction rolls back, the read data was never actually valid.

Q24 **Which isolation level prevents dirty reads, non-repeatable reads, AND phantom reads?**

ADVANCED

A. Serializable

B. Read Committed

C. Read Uncommitted

D. Repeatable Read

Correct Answer: A. Serializable

Explanation: Serializable is the strictest isolation level, effectively executing transactions as if they ran one at a time sequentially, preventing all three anomalies.

Q25 **A 'phantom read' occurs when:**

ADVANCED

A. A transaction re-runs a query and finds new rows that satisfy the condition due to another transaction's insert

B. A transaction reads uncommitted data

C. A row is deleted before being read

D. Two transactions deadlock

Correct Answer: A. A transaction re-runs a query and finds new rows that satisfy the condition due to another transaction's insert

Explanation: Phantom reads happen when a range-based query returns a different SET of rows on re-execution because another transaction inserted or deleted rows matching the condition.

Q26 **What is two-phase locking (2PL) used for?**

ADVANCED

A. Ensuring serializability of concurrent transactions by separating lock acquisition and release phases

B. Encrypting database transactions

C. Load balancing across database replicas

D. Compressing transaction logs

Correct Answer: A. Ensuring serializability of concurrent transactions by separating lock acquisition and release phases

Explanation: 2PL requires a transaction to acquire all needed locks before releasing any (growing phase then shrinking phase), which guarantees conflict-serializable schedules.

Q27 **What is a 'deadlock' in the context of database transactions?** INTERMEDIATE

A. Two or more transactions are waiting indefinitely for locks held by each other

B. A transaction takes too long to execute

C. A transaction is rolled back automatically

D. A table is locked for maintenance

Correct Answer: A. Two or more transactions are waiting indefinitely for locks held by each other

Explanation: A database deadlock occurs when transactions form a circular wait for locks; the DBMS typically detects this and aborts one transaction to break the cycle.

Q28 **What is the purpose of a Write-Ahead Log (WAL)?** ADVANCED

A. To record changes before they are applied to the database, enabling recovery after a crash

B. To log user login attempts

C. To store backup copies of the entire database daily

D. To speed up SELECT queries

Correct Answer: A. To record changes before they are applied to the database, enabling recovery after a crash

Explanation: WAL ensures durability and crash recovery by writing change records to a log before the corresponding data pages are modified on disk.

5. Indexing, Storage & Query Optimization

B-Trees, clustered/non-clustered indexes, query plans

Q29 **What data structure is most commonly used to implement database indexes?** BASIC

A. B+ Tree

B. Linked List

C. Hash Table only

D. Binary Heap

Correct Answer: A. B+ Tree

Explanation: B+ Trees are the standard index structure in most RDBMS because they keep data sorted, support efficient range queries, and remain balanced with logarithmic search time.

Q30 **What is the key difference between a clustered index and a non-clustered index?**

INTERMEDIATE

A. A clustered index determines the physical storage order of table rows (only one per table); a non-clustered index is a separate structure with pointers

B. A clustered index is always slower

C. A non-clustered index can only be used on primary keys

D. There is no functional difference

Correct Answer: A. A clustered index determines the physical storage order of table rows (only one per table); a non-clustered index is a separate structure with pointers

Explanation: A table can have only one clustered index since it dictates the physical row order, but multiple non-clustered indexes, which store pointers back to the actual rows.

Q31 **Why can excessive indexing hurt performance?**

INTERMEDIATE

A. Every INSERT/UPDATE/DELETE must also update all relevant indexes, adding overhead

B. Indexes slow down all SELECT queries

C. Indexes eliminate the need for primary keys

D. Indexes cannot be used with WHERE clauses

Correct Answer: A. Every INSERT/UPDATE/DELETE must also update all relevant indexes, adding overhead

Explanation: While indexes speed up reads, each write operation must also maintain every affected index, so over-indexing increases write latency and storage overhead.

Q32 **What does a query execution plan show?**

ADVANCED

A. The steps and strategy the database engine uses to execute a given query

B. The SQL syntax errors in a query

C. The exact rows returned by a query

D. The users who ran the query

Correct Answer: A. The steps and strategy the database engine uses to execute a given query

Explanation: An execution plan reveals how the optimizer intends to run a query — e.g., which indexes are used, join algorithms, and estimated cost — useful for performance tuning.

Q33 **What is 'covering index' in database optimization?** ADVANCED

A. An index that contains all the columns needed to satisfy a query without accessing the actual table data

B. An index covering all tables in a database

C. An index that never needs to be rebuilt

D. A backup index used during failover

Correct Answer: A. An index that contains all the columns needed to satisfy a query without accessing the actual table data

Explanation: A covering index includes every column referenced by a query (in SELECT, WHERE, etc.), allowing the engine to satisfy the query directly from the index without a costly lookup to the table.

6. NoSQL, Distributed DB & Advanced Concepts

NoSQL types, CAP theorem, sharding, replication

Q34 **Which of these is a document-oriented NoSQL database?** BASIC

A. MongoDB

B. Redis

C. Cassandra

D. Neo4j

Correct Answer: A. MongoDB

Explanation: MongoDB stores data as JSON-like documents (BSON), making it a document-oriented NoSQL database; Redis is key-value, Cassandra is wide-column, Neo4j is graph-based.

Q35 **According to the CAP theorem, a distributed database can guarantee at most how many of Consistency, Availability, and Partition Tolerance simultaneously?** ADVANCED

A. Two

B. All three always

C. One only

D. None, it's theoretical only

Correct Answer: A. Two

Explanation: During a network partition, a system must choose between consistency and availability; you can have at most two of the three guarantees fully at the same time.

Q36 **What is 'sharding' in the context of distributed databases?** INTERMEDIATE

A. Horizontally partitioning data across multiple database servers/nodes

B. Encrypting a database

C. Creating a full backup of the database

D. Vertically splitting a table's columns

Correct Answer: A. Horizontally partitioning data across multiple database servers/nodes

Explanation: Sharding splits a large dataset horizontally (by rows, often via a shard key) across multiple servers to scale storage and throughput beyond a single machine.

Q37 **What is 'eventual consistency' in distributed/NoSQL databases?** ADVANCED

A. If no new updates are made, all replicas will eventually converge to the same value, though they may be temporarily inconsistent

B. All reads always return the most recent write instantly

C. Data is never replicated

D. Consistency is enforced at write time only

Correct Answer: A. If no new updates are made, all replicas will eventually converge to the same value, though they may be temporarily inconsistent

Explanation: Eventual consistency (common in AP systems like DynamoDB/Cassandra) trades immediate consistency for availability, guaranteeing convergence over time rather than instantly.

Q38 **What is database replication primarily used for?** INTERMEDIATE

A. Improving availability and fault tolerance by maintaining copies of data across multiple servers

B. Reducing the size of the database

C. Encrypting sensitive data

D. Normalizing the schema automatically

Correct Answer: A. Improving availability and fault tolerance by maintaining copies of data across multiple servers

Explanation: Replication maintains synchronized copies of data on multiple nodes, improving read scalability, fault tolerance, and disaster recovery.

Q39 **In master-slave (primary-replica) replication, write operations typically go to:** INTERMEDIATE

A. The master/primary node only, then propagate to replicas

B. Any replica node randomly

C. All nodes simultaneously with no primary

D. Only the node nearest the client

Correct Answer: A. The master/primary node only, then propagate to replicas

Explanation: In the classic primary-replica model, all writes are directed to a single primary node, which then propagates changes to read replicas — simplifying consistency for writes.